Project Final Report

Project title: Contract Management System for the Department of Science and Technology

Group members:

- [redacted] - Boyuan Chen
- [redacted] - Rui Cao
- [redacted] - Srinivasa Teja Bharadwaj Penumudi
- [redacted] - Manoj Godavarthi
- [redacted] - Lokesh Kunapaneni

Team number: 8

Project supervisor's name: Dr. Venkat

Industry Partner Name: Department of Science and Technology (DOST), Philippines.

Contact Information: [redacted]

Criteria	High Distinction	Distinction	Credit	Pass	Fail
Project Outcome and success • *Value* • *Useful* • *Complete* **25 marks**	Demonstrates a sophisticated understanding of the project context including the nature of the business or organization and the sector within which it operates The project goals are listed and explained so that their impact is clear. There is a clear and professional assessment of the extent to which the goals were achieved, and explanation provided as to how the assessment was achieved including any metrics use for evaluation.	Demonstrates a high understanding of the project context including the nature of the business or organization and the sector within which it operates The project goals are listed and explained clearly but lack sophisticated engagement and depth of understanding. An evaluation of effectiveness is clearly presented but lacking depth of engagement and inclusion of logical rationale or metric.	Demonstrates a good understanding of the project context including the nature of the business or organization and the sector within which it operates Project goals are listed and an explanation is provided but is missing clear articulation of their impact and value. A general assessment of success is included with limited depth and no inclusion of rationale or evaluative metric.	Demonstrates an understanding of some significant aspects of the project including the nature of the business or industry. The project goals are presented but not explained with depth or detail. Most information is pertinent to the project. A simplistic assessment of success is included with limited, or no rationale or evaluative metric included.	Does not display an understanding of the project correctly, significant aspects missing. Reasoning is deficient. Information is not relevant or flawed. Project goals are not articulated, incorrect or unprofessional in engagement
Project Progress & Reflections • Critical Analysis • Problem Solving • Efficiencies • Effectiveness **25 marks**	• The project increments were executed completely as outlined in the Assessment 2 Project Plan • All increment level evidence on the project progress is included using accepted tools (Trello images, burndown charts, Gantt Charts and retrospective records) • Revisions to the Assessment 2 Project Plan are discussed and justified professionally. • Relevant experiences related to issues your group faced are critically analysed, presented with insightful resolutions	• The project increments were executed mostly as outlined in the Assessment 2 Project Plan • Most increment level evidence on the project progress is included using accepted tools (Trello images, burndown charts, Gantt Charts and retrospective records) • Revisions to the Assessment 2 Project Plan are discussed in detail and justified • Relevant experiences related to group level issues are properly analysed, presented with original resolutions	• The project increments were executed partly as outlined in the Assessment 2 Project Plan • Some of the increment level evidence on the project progress is included using accepted tools (Trello images, burndown charts, Gantt Charts and retrospective records) • Revisions to the Assessment 2 Project Plan are described with limited justifications • Relevant experiences related to group level issues are analysed, presented with basic resolutions	• Most of the project increments did not deliver meaningful outcomes • The increment level evidence on the project progress is not relevant or not included from the tools (Trello images, burndown charts, Gantt Charts and retrospective records, "Done" lists) • Limited Revisions to the Assessment 2 Project Plan are mentioned • Relevant experiences related to group level issues are not discussed	• No useful outcomes were delivered in all increments • The increment level evidence on the project progress is not included from accepted tools (Trello images, burndown charts, Gantt Charts and retrospective records) • Revisions to the Assessment 2 Project Plan are not covered at all • Relevant experiences related to group level issues are not discussed (no group level reflections)

IT artefacts Delivered to the customer • *Quality* • *Professional Standards* **40 marks**	The artefacts and its subcomponents submission fully match with the scope agreed with the client Those are delivered to with high quality standards and well above normal expectations of the industry partner or tutor The IT artefacts are indistinguishable from one produced in a similar time frame by an experienced professional team.	The artefact submission encompasses much of the scope agreed with the client or tutor but falls short in several important areas, with key components either not attempted or completed, or completed well below a professional standard. Overall, the artefact reflects work of a good standard for the work successfully delivered, while remaining below the level as those in the higher-grade band.	The artefact submission broadly reflects the scope agreed with the client, but some aspects of the agreed deliverable may not have been completed or may not have been completed to a professional standard. Overall, the artefact reflects work of a near professional standard for the work successfully delivered, without perhaps reaching the same level as those in the higher-grade band.	The artefact submission encompasses some of the scope agreed with the client but falls well short of an acceptable deliverable for the project. Key components are either not attempted or completed or completed poorly. Overall, the artefact reflects work of a relatively weak standard for the work successfully delivered,	The artefact submission encompasses little or none of the scope agreed with the client or tutor None of the agreed components of the artefact have been delivered, and those mentioned are of poor quality. Key components are either not attempted or poorly.
Communications • *Clarity* • *Conciseness* • *Correctness* **10 marks**	The report is consistently professional in tone and structure and addresses each of the listed requirements in great detail No errors of grammar or structure. Report is organized to aid understanding, and this is assisted by the layout and formatting. The standard of writing exceeds well above expectations	The report is generally professional in tone and structure and addresses each of the listed requirements in detail Very limited errors in grammar or structure. The report is well organized, and the layout and formatting are well chosen. The standard of writing is above expectations	The report is professional in tone and structure but lacks some detail in a small number of the listed requirements. There may be frequent and occasional errors of grammar or structure. Organization and layout remain good, The standard of writing meets the expectations of this level.	The report does not meet expectations, and the coverage is deficient in several the listed requirements. Grammar and structure are variable but are usually ok. The organization is deficient, but some effort has been made to structure and format the document. The standard of writing may lie below the expectations at this level	The report doesn't meet the requirements set out in the brief. Sections missing or poorly covered. Meaning unclear as grammar and/or spelling contain frequent errors Disorganised or incoherent writing Structure either absent or incoherent and the standard of writing may be well below the expectations at this level

Executive Summary:

The Department of Science and Technology (DOST) relied on a manual, fragmented contract management process with no centralised tracking system. This approach created significant inefficiencies in contract oversight, approval workflows, and lifecycle visibility, posing risks to operational accountability and compliance across the organisation.

To address this, our team designed and developed a web-based Contract Management System built on a Laravel REST API backend, React frontend, and MySQL database. The system delivers three role-based access tiers (Technical Admin, Functional Admin, and Staff), ensuring appropriate separation of duties across all user interactions. Key modules delivered include a Contract Repository with full lifecycle management (Draft → Pending → Active/Rejected, where Pending contracts may additionally require FA Review before progression), automated contract expiry handling, a multi-step approval workflow, automated email notifications for submissions, approvals, rejections, and expiry alerts, comprehensive audit logging across all system actions, and a real-time dashboard providing instant contract status visibility. Together, these features provide DOST with a centralised, transparent, and auditable platform to manage their contractual obligations end-to-end.

Throughout the project, the team navigated three key challenges. First, requirements alignment required ongoing stakeholder synchronisation meetings to ensure complex feature expectations were accurately captured and minimise rework. Second, API and integration consistency was maintained through rigorous API documentation and early-stage integration testing to prevent data format mismatches between the frontend and backend. Third, scope management was enforced via a strict change-control process, allowing the team to evaluate and contain potential scope creep without disrupting the delivery timeline.

Table of Contents

Analysis: *6*

Problem Identification 6

Requirements Analysis 6

Technical Solution Analysis 7

Project Management and Risk Analysis 7

Design: *7*

System Architecture 7

Database Design 8

Role-Based Access Control Design 9

Contract Lifecycle Design 10

Approval Workflow Design 11

Email Notification and Dashboard Design 11

Project Progress and Reflections *12*

Project Increment Execution 12

Revisions to the Project Plan 17

Outcomes *18*

Project Outcomes and Goal Achievement 18

Testing and Quality Assurance 18

Implementation Considerations 19

Group Reflection *20*

Learning New Tools and Technologies 20

Experience of Working with an Industry Partner 21

Handling Changes During the Project 21

Analysis:

Problem Identification

The project was initiated in response to DOST's reliance on a manual and fragmented contract management process, which lacked any centralised system for tracking contract status, managing approvals, or maintaining an auditable record of actions. This created operational inefficiencies and posed risks to organisational accountability and compliance. To gain a thorough understanding of the problem, the team engaged with DOST stakeholders through multiple channels, including WhatsApp communication, formal meetings, and structured question lists. This multi-channel approach allowed the team to progressively clarify the scope of the problem and ensure that the identified requirements accurately reflected the organisation's operational needs.

Requirements Analysis

Through early stakeholder engagement, the team identified the need for a role-based system that could enforce separation of duties across different user types. However, the initial requirements did not fully capture the complexity of DOST's internal structure. During a mid-project meeting, the client indicated a desire for more granular role definitions and a more elaborate approval workflow, reflecting the organisation's departmental access requirements. Given the stage of development at which this change was raised, the team critically evaluated the feasibility of fully implementing the requested complexity within the remaining timeline. A decision was made to rationalise the role structure into three tiers — Technical Admin, Functional Admin, and Staff — which preserved the core intent of the client's request while remaining achievable within the project constraints. This required balancing client expectations against delivery risk, which is a key aspect of critical thinking in project execution.

Technical Solution Analysis

The technology stack was partly determined by client requirements, as DOST specified Laravel as the backend framework. To complement this and match the team's development capability, React was selected as the frontend framework and MySQL as the database. This combination provided a clear separation between frontend and backend concerns, enabling parallel development across the team. To mitigate the risk of data format mismatches during integration, the team proactively developed comprehensive API documentation that both the frontend and backend developers adhered to strictly. This preventative measure proved effective, as no significant integration issues were encountered during development.

Project Management and Risk Analysis

The team managed the project using Jira for task tracking, GitHub for version control, and Google Shared Docs for collaborative documentation. These tools provided visibility into individual contributions and supported structured progress tracking throughout the project lifecycle. The primary risk that materialised was related to requirements alignment, as the mid-project change request from the client introduced scope challenges. This was partly attributed to insufficient early-stage communication with the client, which meant that the full complexity of their requirements was not surfaced until later in the project. In response, the team implemented a change-control process, evaluating each new request through team discussion before committing to any additional development, ensuring that the core deliverables remained on track.

Design:

System Architecture

The system follows a standard three-tier client-server architecture with full separation between the frontend and backend. The React frontend communicates exclusively with the Laravel backend through RESTful API calls,

while Laravel handles all business logic and interacts with the MySQL database. This separation allowed frontend and backend developers to work in parallel during development, with shared API documentation serving as the agreed interface between both sides.

Database Design

The database was structured around a set of core tables designed to support the system's business logic, security requirements, and auditability. The users table incorporates security-oriented fields including failed login attempt tracking, account lock timestamps, and password change history, reflecting a deliberate focus on access control at the data layer. The contracts table serves as the central entity, storing contract metadata, file references, lifecycle status, and department ownership. A versioning mechanism was implemented through a version field and a self-referencing parent ID foreign key, allowing revised contracts to be linked to their originals while maintaining a full revision history. Soft deletion was applied to ensure no contract records are permanently removed from the system.

The approval workflow is supported by two complementary tables. The approval templates table defines the default approval chain for each contract type, specifying the sequential step order, role designation, and default assignee. This allows the system to automatically pre-populate the approval workflow when a new contract is created, reducing manual configuration. The contract approvals table then captures each live approval step for individual contracts, recording both the originally proposed approver and any subsequently reassigned approver, preserving a complete chain of custody for every decision. Changes to approval template configurations are independently tracked in the allocation rule logs table, which stores the full before and after state of each template modification as JSON snapshots, providing a dedicated audit trail specifically for workflow rule changes. The audit logs table captures all system actions including pre-authentication events such as failed login attempts, with each entry recording the action, affected module, target entity, old and new values, IP address, and user agent. The notifications table stores in-system alerts on a per-user basis with read status tracking, complementing the email notification channel.

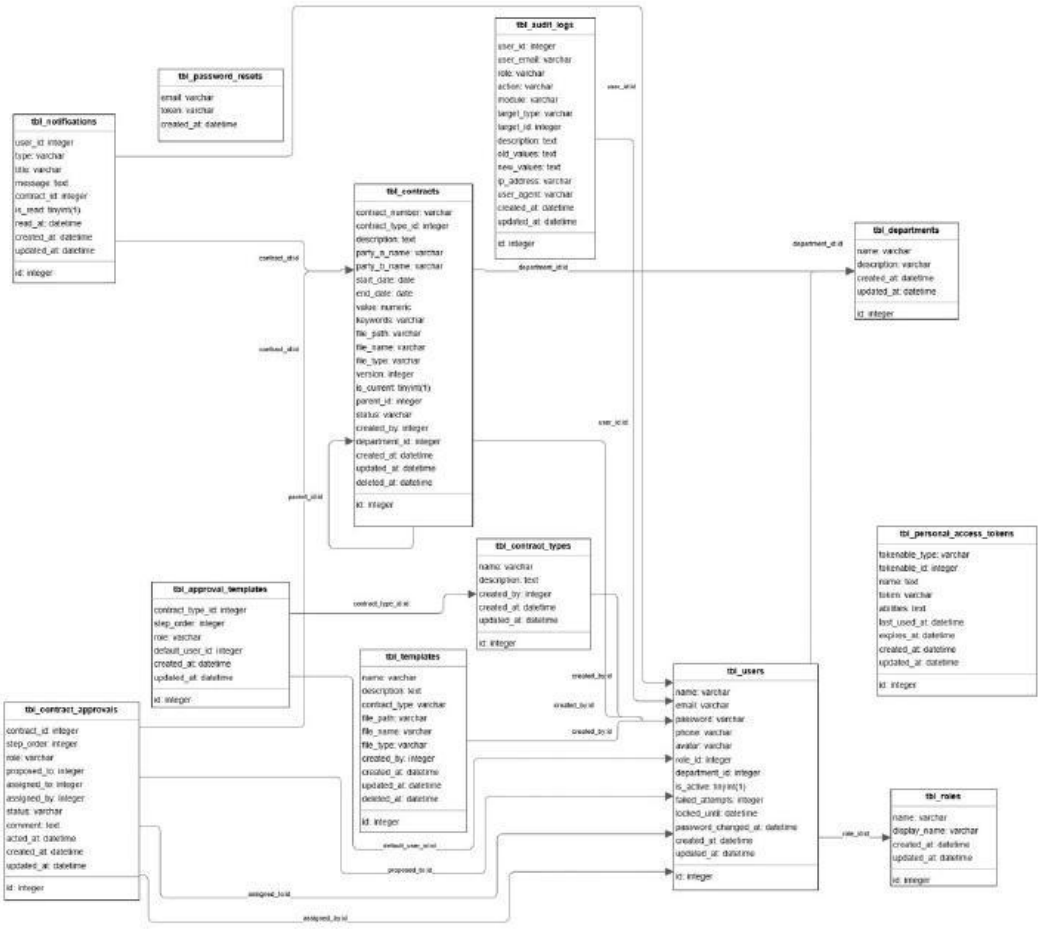


Figure 1: Entity Relationship Diagram

Role-Based Access Control Design

A core design decision was the implementation of three distinct user roles: Technical Admin, Functional Admin, and Staff. Each role operates within a strictly defined permission boundary enforced at both the frontend and backend layers. Technical Admin handles system-level operations such as user management and audit trail access, and is explicitly denied access to contract data. Functional Admin has full visibility over all contracts and oversees the review and approval workflow. Staff can create and manage their own contracts and view active contracts submitted by others. This separation of

duties was designed to reflect DOST's internal operational structure and ensure no single user has unchecked authority over the contract lifecycle.

Role	Contract visibility	Permissions
Technical Admin	✗ No access to any contracts	✓ User CRUD, activate / deactivate ✓ View roles & departments ✓ Full audit log access ✗ No contract or approval actions
Functional Admin	✓ All non-DRAFT contracts, including all historical versions Cannot see anyone's DRAFTs	✓ Contract type CRUD ✓ Approval template management ✓ Assign / reassign pending steps ✓ Audit log (contract-related only) ✗ Cannot create, edit or approve contracts
Staff	✓ Own contracts — all statuses, latest version ✓ Assigned contracts — current step only (PENDING) ✓ All ACTIVE & EXPIRED contracts — latest version only Cannot see others' DRAFT / PENDING / REJECTED No access to version history	✓ Create, edit, delete own contracts (DRAFT only) ✓ Submit & resubmit (creates new version) ✓ Review / approve when assigned as current step ✗ Cannot approve own contracts (SOD) ✗ Cannot participate more than once in the same contract

Figure 2: Role-Based Access Control Matrix

Contract Lifecycle Design

The contract lifecycle was designed to accommodate multiple workflow paths. A contract begins in Draft status, where it can be edited or deleted by its creator. Upon submission it transitions to Pending status, which may escalate to Pending FA Review if a conflict of interest is identified or if the submitter has designated specific approvers requiring FA oversight. Active contracts are monitored continuously and expiry is handled automatically by the system. Rejected contracts are returned to Draft status, allowing the creator to revise and resubmit. Functional Admins retain the ability to proactively intervene at any stage, ensuring operational oversight is never blocked by workflow constraints.

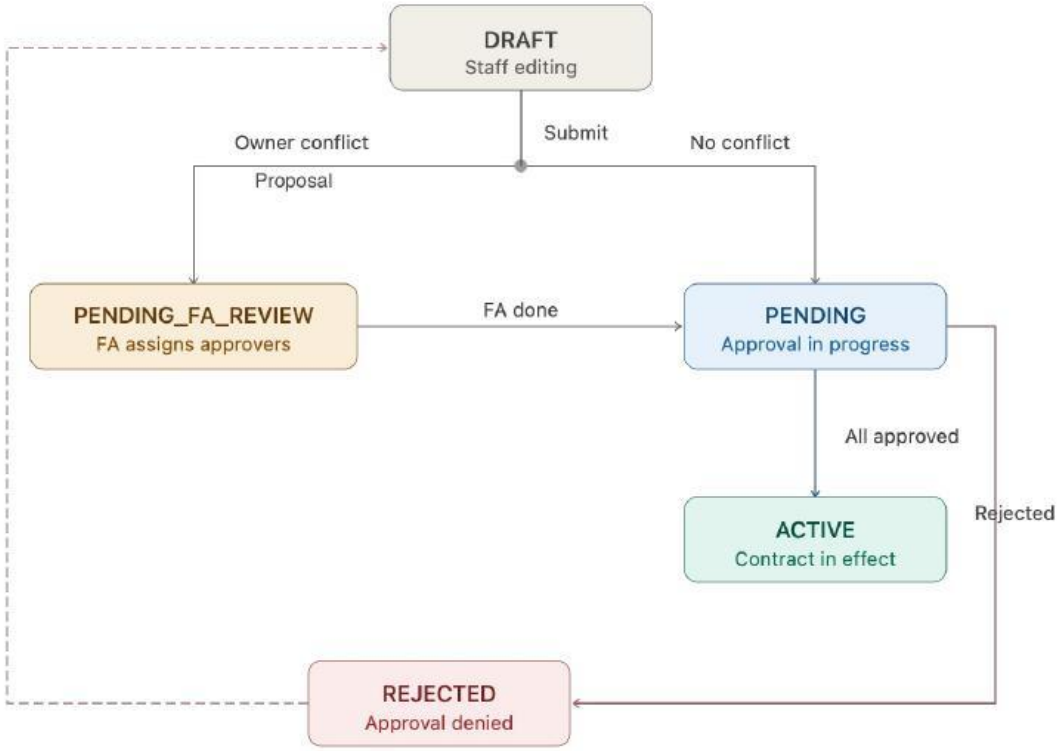


Figure 3: Contract Lifecycle State Diagram

Approval Workflow Design

The multi-step approval workflow enforces separation of duties by ensuring the staff member who submits a contract cannot also approve it. Each approval step is ordered sequentially and assigned a role of either reviewer or approver. When a new contract is created, the system automatically pre-populates the approval chain based on the approval template configured for that contract type, with each step inheriting the designated default approver. The workflow records both the originally proposed approver and any reassigned approver, maintaining a traceable chain of custody. Every action within the workflow is captured in the audit log with a timestamp and the responsible user.

Email Notification and Dashboard Design

Email notifications were implemented using Laravel's built-in Mail system through a centralised notification class that handles all notification types across the contract lifecycle, including submissions, approvals, rejections, and expiry

alerts. Each notification email contains a direct link to the relevant contract, enabling recipients to act immediately. The dashboard complements this by presenting role-differentiated views: Functional Admins see a real-time contract status breakdown, an expiring contracts panel with colour-coded urgency indicators, and a recent contracts list; Technical Admins see system-level information including user counts and recent audit entries; Staff see a personalised view scoped to their own contracts.

Project Progress and Reflections

Project Increment Execution

The project was executed across seven sprints following an Agile development methodology. All seven objectives were delivered within their planned timeframes, as evidenced by the Jira backlog and timeline records below.

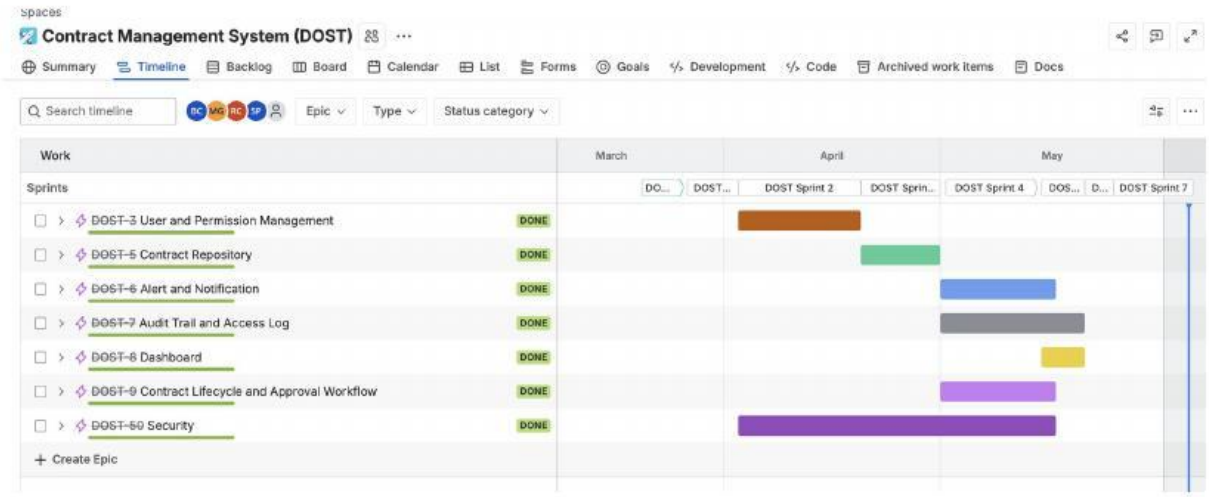


Figure 4: Jira Project Timeline

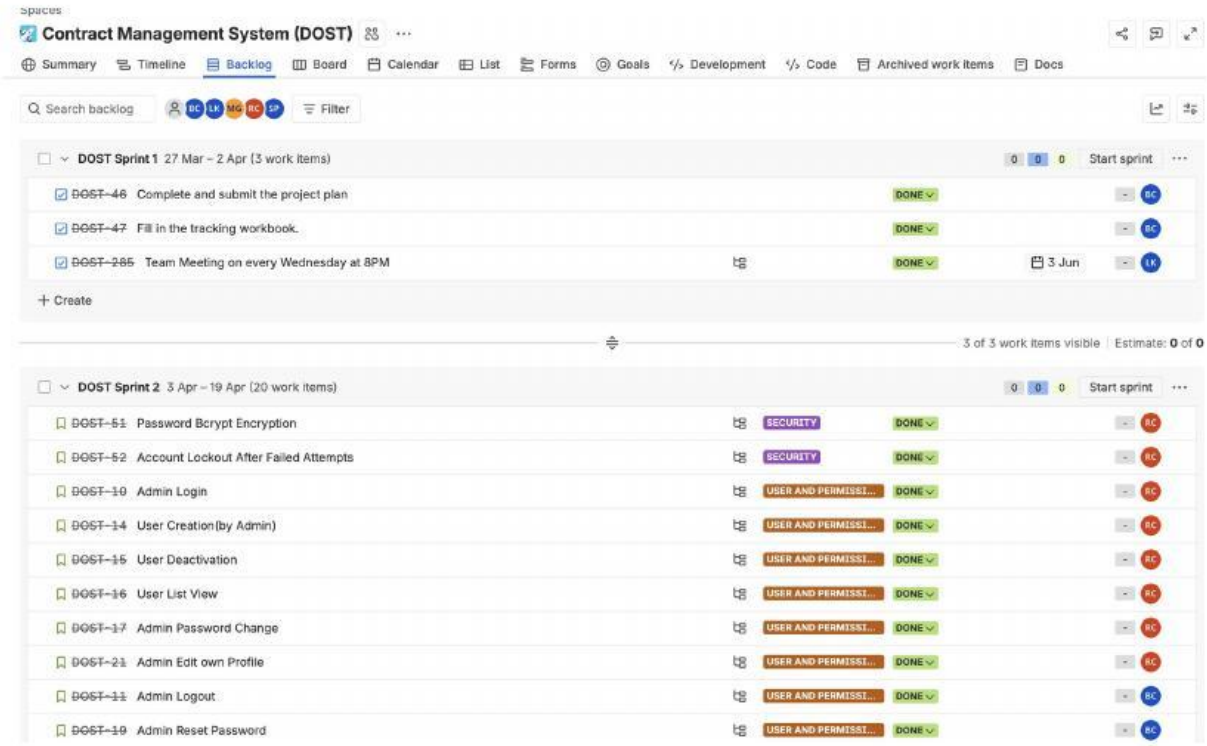


Figure 5: Jira Backlog — Sprint 1 to Sprint 2

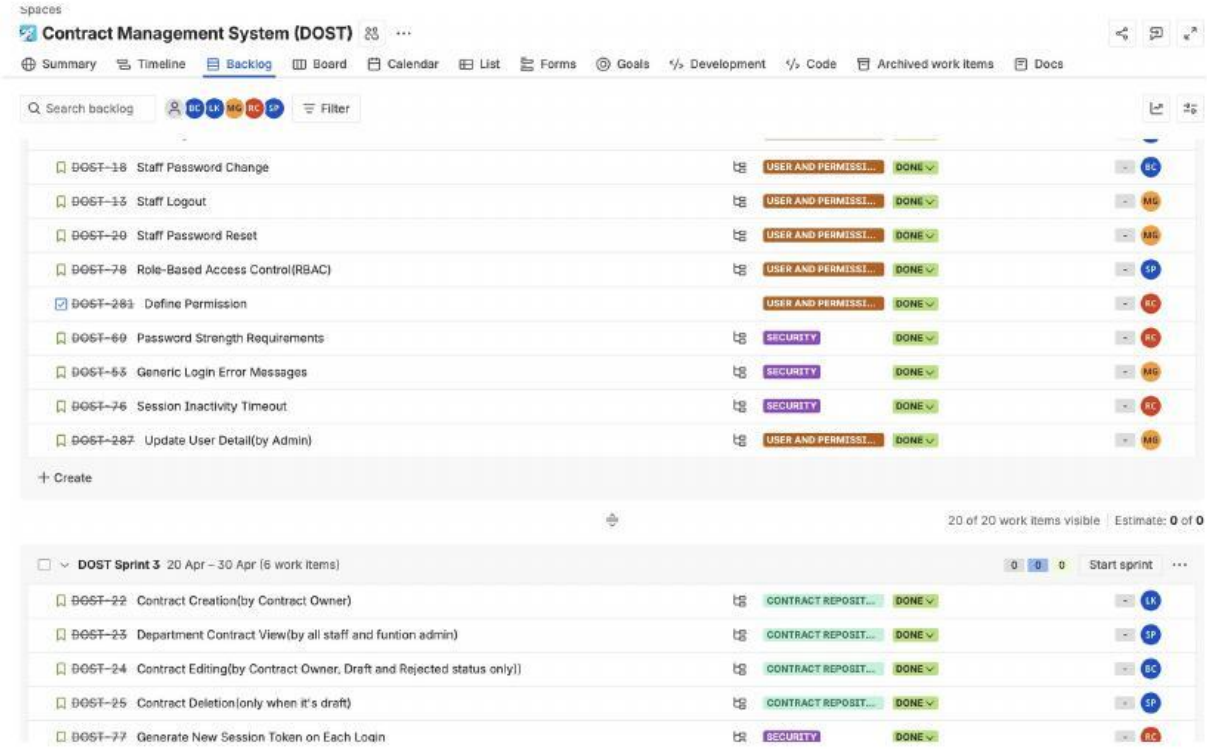


Figure 6: Jira Backlog — Sprint 2 to Sprint 3

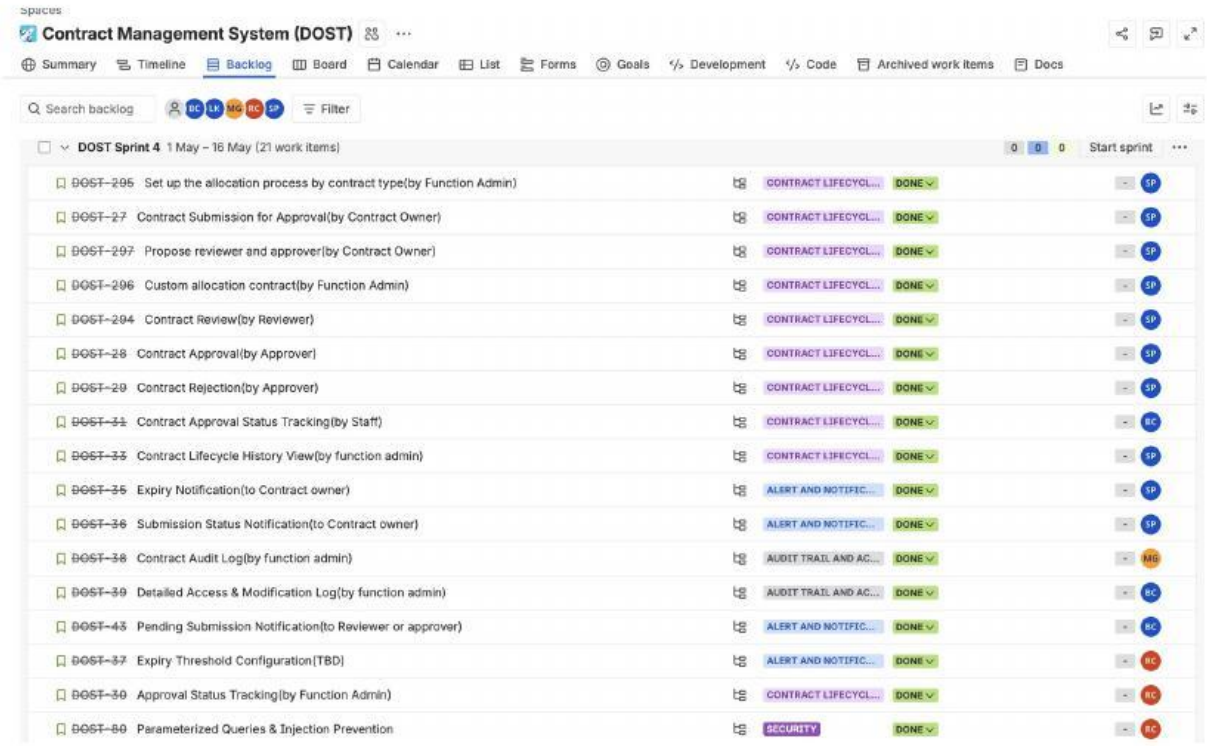


Figure 7: Jira Backlog — Sprint 4

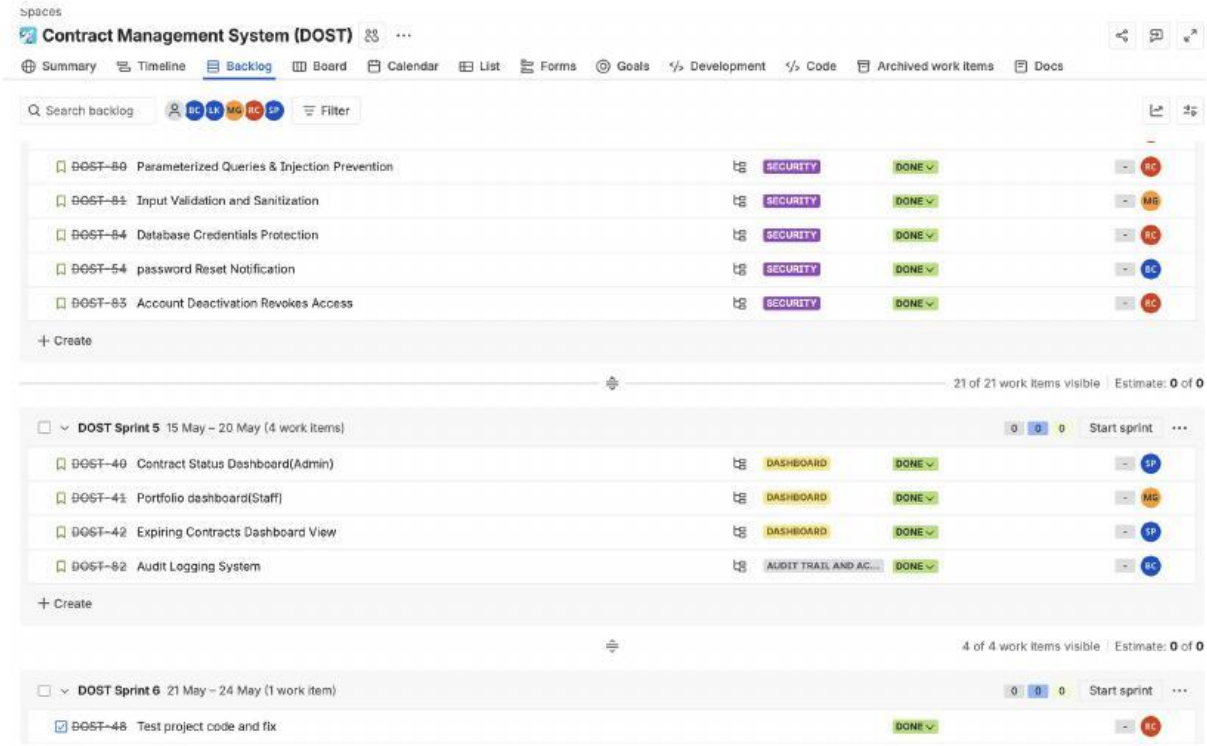


Figure 8: Jira Backlog — Sprint 4 to Sprint 6

Sprint 1 (27 March – 2 April) completed project initiation tasks including plan submission, tracking workbook setup, and initial stakeholder engagement with DOST to inform scope definition. Sprint 2 (3 April – 19 April) delivered the User and Permission Management module, completing all 20 work items including authentication, role-based access control, and a comprehensive set of security features. Sprint 3 (20 April – 30 April) delivered the Contract Repository module across 6 work items, covering contract creation, editing, deletion, and department-based access controls.

Sprint 4 (1 May – 16 May) was the most demanding sprint, comprising 21 work items across the Contract Lifecycle and Approval Workflow, Alert and Notification, Audit Trail, and Security modules. The high workload reflected two compounding factors: the approval workflow carried inherent technical complexity due to its multi-step sequential structure, role-based assignments, and chain-of-custody audit requirements; and a mid-sprint requirement change from DOST introduced additional scope pressure. The team responded by convening an internal triage meeting, rationalising the role structure to three tiers, and adopting a change-control process for the remainder of the project. All 21 work items were completed without timeline slippage. Internal meeting records from this period confirm the team's coordinated response to the requirement change, with role definitions, permission boundaries, and outstanding frontend tasks explicitly documented and assigned as action items.

Quick recap

The team discussed progress on their project implementation, focusing on frontend and backend developments. Manoj reported completing audit logs, notifications, and approval workflows in the frontend, while Aria confirmed recent backend updates including approval and audit log functionality. The team clarified role-based permissions, with Technical Admins able to manage users and Functional Admins able to view contract-related audit trails. Key remaining tasks identified included implementing contract types dropdown functionality and resolving issues with contract approval workflows showing in the frontend. The team agreed Manoj would demonstrate the complete frontend system at tomorrow's client meeting, with Aria and Boyuan providing backend support as needed.

Next steps

- Aria: After the meeting, write in the group the exact permissions and dashboard components accessible to Staff 1, Staff 2, and Staff 3 for Manoj to replicate in the frontend.
- Manoj: Implement and display the "contract type" functionality in the frontend, including the ability for functional admin to create/manage contract types and for staff to select contract types when creating contracts, as per the screenshots and descriptions provided by Aria and Boyuan.
- Manoj: Investigate and resolve the issue where newly created contracts are not appearing for functional admin in the frontend, and ensure audit trail for contracts is visible to functional admin (but not for user management actions).
- Manoj: Demonstrate the completed frontend implementation (including contract type, contract approval, notifications, and audit log) in the meeting with Kathy tomorrow.
- Aria: Provide Manoj with any additional documentation or screenshots as needed to clarify functional vs technical admin roles and contract type management.
- Manoj: Send a screenshot of the completed contract type page to Aria for confirmation before the client meeting.
- Boyuan: If Manoj is unable to complete the contract type functionality as specified, be prepared to complete it before the client meeting.

Summary

Frontend Backend Update Implementation Review

Manoj reported updating the frontend to replicate backend changes, including adding an audit trail filter for months and showing real-time logs with a limit of 20 per page. Aria confirmed seeing the updates in the backend and expressed approval of the implementation.

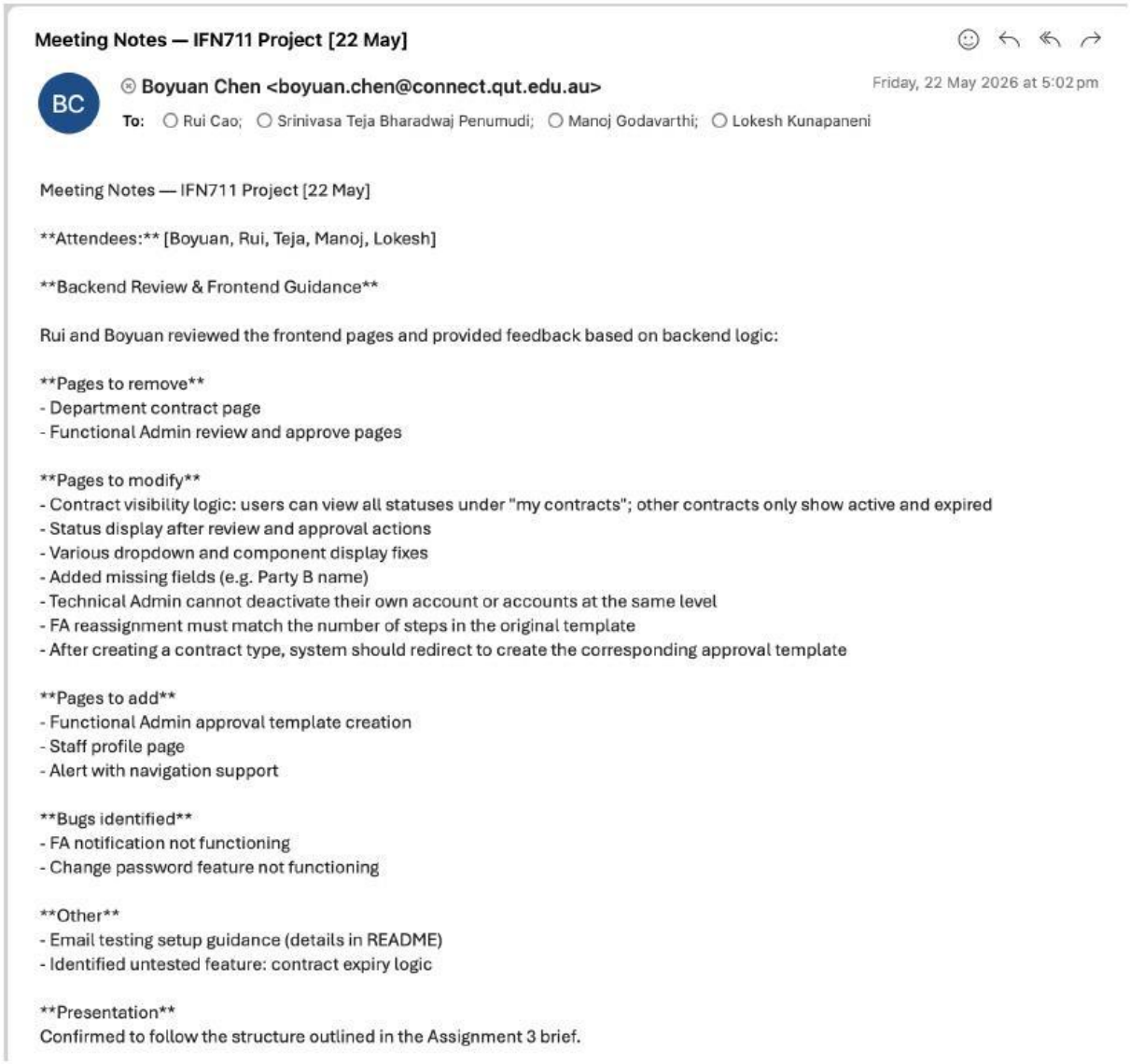
Role-Based Controls Implementation Discussion

Manoj and Aria discussed the implementation of role-based controls for three staff users (Staff 1, Staff 2, and Staff 3). Aria confirmed that they had pushed code to GitHub the previous day, including features for

Figure 9: Sprint 4 Internal Meeting Notes

Sprint 5 (15 May – 20 May) delivered the Dashboard module across 4 work items. A formal team review was conducted on 22 May, with findings documented and circulated via email, identifying two bugs and several UI modifications that directly defined the Sprint 6 scope. Sprint 6 (21 May – 24

May) addressed all identified issues and was completed slightly ahead of schedule. Sprint 7 (25 May – 4 June) covered final documentation and the formal handover to DOST.



Meeting Notes — IFN711 Project [22 May]

BC

Boyuan Chen <boyuan.chen@connect.qut.edu.au>

Friday, 22 May 2026 at 5:02 pm

To: Rui Cao; Srinivasa Teja Bharadwaj Penumudi; Manoj Godavarthi; Lokesh Kunapaneni

Meeting Notes — IFN711 Project [22 May]

Attendees: [Boyuan, Rui, Teja, Manoj, Lokesh]

Backend Review & Frontend Guidance

Rui and Boyuan reviewed the frontend pages and provided feedback based on backend logic:

Pages to remove
- Department contract page
- Functional Admin review and approve pages

Pages to modify
- Contract visibility logic: users can view all statuses under "my contracts"; other contracts only show active and expired
- Status display after review and approval actions
- Various dropdown and component display fixes
- Added missing fields (e.g. Party B name)
- Technical Admin cannot deactivate their own account or accounts at the same level
- FA reassignment must match the number of steps in the original template
- After creating a contract type, system should redirect to create the corresponding approval template

Pages to add
- Functional Admin approval template creation
- Staff profile page
- Alert with navigation support

Bugs identified
- FA notification not functioning
- Change password feature not functioning

Other
- Email testing setup guidance (details in README)
- Identified untested feature: contract expiry logic

Presentation
Confirmed to follow the structure outlined in the Assignment 3 brief.

Figure 10: Sprint 5 Retrospective Meeting Notes

Revisions to the Project Plan

The overall execution closely followed the Assessment 2 project plan. The one notable revision occurred during Sprint 4, as described above. Despite this change, no objectives were delayed and no planned deliverables were descoped. A change-control process was adopted from Sprint 4 onwards to prevent further unplanned scope additions.

Outcomes

Project Outcomes and Goal Achievement

The primary objective of this project was to replace DOST's manual, fragmented contract management process with a centralised, auditable, and role-based digital system. The agreed scope comprised seven functional modules: User and Permission Management, Contract Repository, Contract Lifecycle and Approval Workflow, Alert and Notification System, Audit Trail, Dashboard, and Security. All seven modules were delivered in full and accepted by the client at the formal handover meeting.

Goal achievement was assessed against three criteria. First, functional completeness: all 55 planned work items across Sprints 1 to 6 were completed and marked as Done in Jira, with zero items descoped or deferred. Second, defect resolution: all defects identified during manual functional testing and API testing were resolved prior to final delivery, including the two bugs surfaced in the Sprint 5 retrospective review. Third, client acceptance: a formal demonstration was conducted with DOST, during which all system functionalities were presented and the completed system and documentation were handed over without outstanding issues raised by the client.

Against the original problem statement, the delivered system directly addresses each identified inefficiency: contract status is now tracked centrally in real time via the dashboard, approval workflows are enforced digitally with a full audit trail, and role-based access control ensures separation of duties across all user interactions.

Testing and Quality Assurance

To validate the reliability of the delivered system, the team conducted two forms of testing throughout the development process: manual functional testing and API testing via Postman.

Manual functional testing was performed across all core modules, including contract creation and lifecycle management, the multi-step approval workflow, role-based access control, email notifications, and the dashboard. Each feature

was tested by navigating through the system as different user roles to verify that permission boundaries were correctly enforced and that the expected behaviour was produced at each stage of the contract lifecycle. This approach allowed the team to identify and resolve a range of defects during development, including issues with field validation, incorrect function logic, and edge cases in the approval workflow. All identified defects were resolved prior to final delivery.

API testing was conducted using Postman to verify that each endpoint returned the correct response structure, status codes, and data under both valid and invalid input conditions. This was particularly important given that the frontend and backend were developed in parallel, and ensuring API contract consistency was critical to preventing integration issues.

Security-related functionality was also verified as part of the testing process. This included validating that login authentication functioned correctly, that account lockout was triggered after repeated failed login attempts, that session tokens were properly managed, and that role-based access control prevented users from accessing functionality outside their permitted scope. These checks confirmed that the security measures embedded in the system design were functioning as intended.

No automated test suite was implemented within the project timeline. However, the manual and API testing activities were sufficient to validate the core functionality of the system against the agreed requirements.

Implementation Considerations

Although the system was developed and validated in a local development environment, it has been designed with production deployment in mind. The following considerations are provided to assist DOST in transitioning the system to a live environment.

The system requires a server environment capable of running a Laravel backend and a React frontend, with a MySQL database instance. Standard web hosting infrastructure or a cloud-based platform would be suitable for deployment. A production environment configuration, including environment

variables for database credentials, mail server settings, and the frontend URL, will need to be established prior to deployment.

A formal handover was conducted with DOST through a meeting in which all system functionalities were demonstrated and explained. In addition, the team produced project documentation to support onboarding and future maintenance, enabling DOST's technical staff to understand the system architecture and operational procedures without requiring direct involvement from the development team. It is further recommended that DOST conduct a user acceptance testing phase prior to going live, and provide role-based training to staff members to ensure familiarity with the system's workflows and permission structure.

Group Reflection

Learning New Tools and Technologies

At the outset of the project, the client specified that the backend must be developed using PHP with the Laravel framework. Several team members had no prior experience with PHP or Laravel, which presented an immediate learning challenge alongside the existing project workload. Unlike university assignments where teams are typically free to choose familiar technologies, this project replicated a real industry environment where technical decisions are often driven by client requirements rather than developer preference.

In response, affected team members independently invested time in learning Laravel through tutorials, official documentation, and hands-on experimentation. Key concepts including routing, database migrations, Artisan commands, and API development had to be understood quickly in order to contribute effectively to the project. The team also adopted Jira for task management and GitHub for version control and collaborative development, tools that some members were using in a structured team environment for the first time.

Reflecting on this experience, the team recognised that adaptability is one of the most important skills in professional software development. The process of

learning Laravel also highlighted how transferable programming knowledge can be — core concepts such as API logic, database operations, and application structure follow similar principles across different frameworks, which made the learning curve more manageable than initially anticipated. This experience reinforced the team's confidence in approaching unfamiliar technologies in future projects.

Experience of Working with an Industry Partner

Working with DOST as an industry partner provided the team with a significantly different experience compared to standard university projects. Communication took place through WhatsApp, structured meetings, and question lists, which required the team to maintain professional and timely correspondence throughout the project.

One of the key learnings from this experience was that industry clients often cannot fully articulate their requirements in the early stages of a project. During initial discussions, DOST provided general descriptions of their needs, but many important business logic details only became clear as the project progressed and working features could be demonstrated. This taught the team that effective communication with an industry partner requires more than simply receiving a brief — it requires ongoing dialogue, proactive clarification, and the use of visual demonstrations to surface misunderstandings early.

The team also learned the importance of managing client relationships professionally. When expectations needed to be adjusted or timelines discussed, clear and respectful communication helped maintain a productive working relationship with DOST throughout the project. The final handover meeting, where the completed system was demonstrated and project documentation was delivered, reinforced the value of transparency and preparation in client-facing interactions.

Handling Changes During the Project

The most significant challenge the team faced was a mid-project change in requirements. During a meeting with DOST, the client indicated the need for a

more granular role structure and a more complex approval workflow than what had been initially scoped. This change required the team to revisit and revise work that had already been completed, creating additional pressure during a critical stage of development.

The team responded by increasing the frequency of internal communication and redistributing tasks to accommodate the new requirements. Decisions about what could be realistically implemented within the remaining timeline were made collaboratively, resulting in a rationalised role and workflow design that preserved the intent of the client's request while remaining deliverable. A stricter change-control process was also adopted from that point forward, requiring all new requests to be discussed and agreed upon by the team before any development work was committed.

Reflecting on this experience, the team acknowledged that earlier and more frequent engagement with the client during the initial stages could have reduced the scale of the mid-project change. If working prototypes or interface demonstrations had been shared with DOST sooner, the workflow and role requirements might have been identified before significant development had already taken place. This remains the team's key takeaway for future industry projects: requirement clarity is an ongoing process, and visual demonstrations are one of the most effective tools for aligning client expectations with technical implementation.